A Deep Reinforcement Learning Approach for Joint Task Offloading, Caching, and User Association in Mobile Edge Computing

Tasnia Ashrafi Heya Sayed Erfan Arefin

Department of Computer Science, University of Dayton

10th International Conference on Fog and Mobile Edge Computing (FMEC) 2025

Introduction & Motivation

- Modern mobile apps (e.g., AR/VR, 3D games, recognition) are computation-heavy, while devices remain resource constrained.
- Mobile Edge Computing (MEC) pushes computation from devices to nearby edge nodes (APs / SBSs) to reduce latency.
- However, simultaneous offloading from many users causes:
  - Highly dynamic edge loads,
  - Congestion and deadline violations,
  - Inefficient use of limited radio and compute resources.
- We focus on **jointly** deciding:
  - Whether to offload a task or process locally,
  - Which AP(s) should serve each user,
  - Which files to cache / evict at each AP.

Problem Statement

- Edge nodes have limited storage, bandwidth, and compute.
- Pure task-offloading policies that ignore caching and AP load lead to:
  - Long transmission + queuing delay,
  - High task drop ratios when queues overflow.
- Existing DRL and game-theoretic methods often:
  - Assume single AP association per user,
  - Require global knowledge of the entire network state,
  - Do not scale well in ultra-dense MEC deployments.
- **Goal:** Learn a *decentralized* policy that coordinates caching, user–AP association, and task offloading to minimize end-to-end delay.

Key Contributions

- A **Deep Q-Network (DQN)** framework that couples:
  - User–AP association,
  - Cache placement / replacement,
  - Task offloading decisions.
- Allows each user to associate with **multiple APs** to fetch different files.
- Incorporates channel state information (CSI), interference, and cache status into the DRL state.
- Uses **Zipf** for initial caching and **Bernoulli** association to model realistic content requests.
- Demonstrates improved convergence, reduced delay, and lower task drop ratio compared with a state-of-the-art DRL baseline.

Related Work

- DRL-based offloading:
  - Deep RL for local/offload decisions and edge scheduling.
  - DQN for offloading with large state spaces and wireless charging.
- Caching replacement:
  - Greedy caching under matroid constraints for delay minimization.
  - DQN-based caching focusing mainly on delay but not multi-AP association.
- Game-theoretic models:
  - Weakly acyclic games and Two-Dimensional Nash Equilibria (2D-NE) for caching and MU–BS association.
- **Gap:** Existing methods typically treat offloading, caching, and association separately and often rely on centralized knowledge.

References

[1] F. Guo et al., "Joint optimization of caching and association in energy-harvesting-powered small-cell networks," *IEEE Trans. Veh. Tech.*, 2018.

[2] M. Tang and V. W. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, 2022.

[3] X. Chen et al., "Performance optimization in mobile-edge computing via deep reinforcement learning," arXiv:1804.00514, 2018.

[4] J. Wang et al., "Smart resource allocation for mobile edge computing: A deep reinforcement learning approach," *IEEE TETC*, 2021.

(Full bibliography available in the paper.)

System Model

Network:

- B APs (edge nodes), set $\mathcal{B} = \{1, \dots, B\}$.
- K mobile users, set $\mathcal{K} = \{1, \dots, K\}$.
- Content library with F files; each AP caches a subset of $J \ll F$ files.

Connectivity:

- Binary user–AP connectivity matrix $P(t) \in \{0,1\}^{K \times B}$: $P_{bk}(t) = 1$ if user k is in coverage of AP b.
- Binary caching matrix $Z(t) \in \{0,1\}^{F \times B}$: $Z_{fb}(t) = 1$ if file f is cached at AP b.

Requests:

- Each user requests R files, modeled by request vectors $u_{bk} \in \{0,1\}^F$, with $\|u_{bk}\|_1 = R$.
- Request popularity follows a Zipf distribution.

Wireless Model & Delay

Downlink rate (single AP):

$$R_{bk}(t) = W_{bk} \log_2 \left(1 + \frac{P_{bk}|h_{bk}(t)|^2}{N_0 + I_k}\right)$$

- W_{bk}: bandwidth share for user k at AP b.
- P_{bk}: transmit power; $h_{bk}(t)$: channel gain;
- N_0: noise PSD; I_k: interference from other APs.

Delay components:

- If file is cached at AP: delay $\propto \dfrac{|s_f|}{R_{bk}(t)}$.
- If file is not cached: additional backhaul / fetching penalty D_{cf}.

Cost (per AP b):

$$c_{3,t,b} = \sum_{f,k} \sigma_{bk}(t)\alpha_{bf}(t)u_{bfk}\frac{|s_f|}{R_{bk}(t)} + \sum_{f,k} \sigma_{bk}(t)(1-\alpha_{bf}(t))$$

where $\sigma_{bk}(t)$ is the association indicator and $\alpha_{bf}(t)$ is the caching indicator.

DRL Formulation (Per AP b)

State:

$$x_b(t) = (\alpha_b(t), \sigma_b(t), G_b(t))$$

- $\alpha_b(t)$: cache state (F-dimensional binary vector),
- $\sigma_b(t)$: user–AP association (K-dimensional binary vector),
- $G_b(t)$: CSI / channel quality indicators.

Action:

$$y_b(t) = (\sigma_b(t), \alpha_b(t))$$

- Joint decision: which users to serve and which files to cache/evict.

Reward:

$$r_b(t) = -c_{3,t,b}$$

- Negative total delay encourages low-latency policies.

Objective:

- Learn Q-network parameters θ to minimize discounted long-term cost:

$$\min_{\pi} \ \mathbb{E}\left[\sum_t \gamma^t c_{3,t,b}\right]$$

Enhanced DQN Algorithm

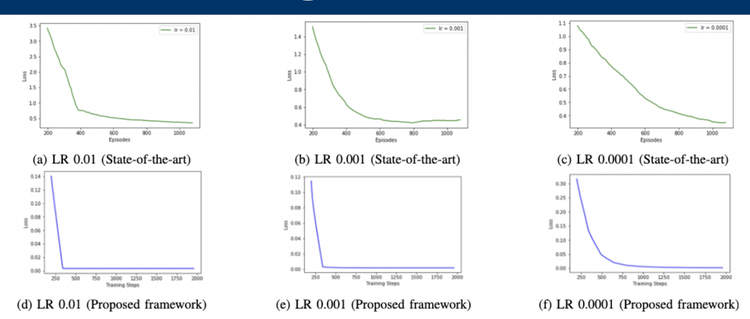
Key Features

- Replay buffer with capacity limit and mini-batch sampling.
- Separate **primary** and **target** Q-networks with periodic soft/hard updates.
- ϵ-greedy exploration with initial ϵ, minimum $\epsilon_{\min}$, and decay factor.
- Actions jointly encode user–AP associations and cache replacement.

High-Level Loop

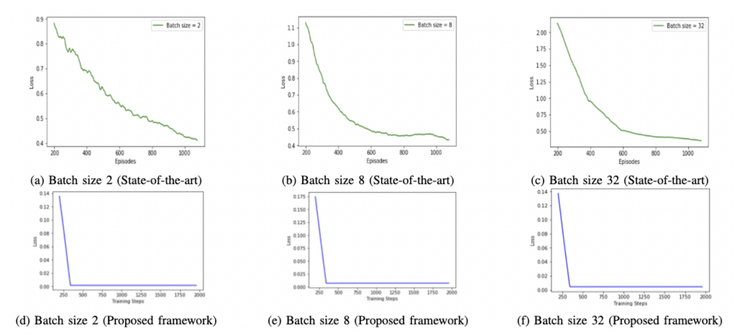
1. Observe current state $x_b(t)$ (cache, associations, CSI).
2. Choose action $y_b(t)$ via ϵ-greedy over Q-values.
3. Apply action: update cache and association; compute delay-based cost.
4. Store $(x_b(t), y_b(t), r_b(t), x_b(t+1))$ in replay buffer.
5. Sample mini-batch, compute targets, and update θ by gradient descent.
6. Periodically update target network parameters and decay ϵ.

Results: Convergence Behavior



(a) LR 0.01 (State-of-the-art) (b) LR 0.001 (State-of-the-art) (c) LR 0.0001 (State-of-the-art)
(d) LR 0.01 (Proposed framework) (e) LR 0.001 (Proposed framework) (f) LR 0.0001 (Proposed framework)

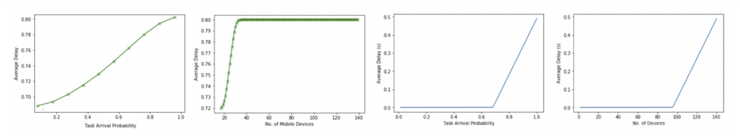
Convergence under different learning rates.



(a) Batch size 2 (State-of-the-art) (b) Batch size 8 (State-of-the-art) (c) Batch size 32 (State-of-the-art)
(d) Batch size 2 (Proposed framework) (e) Batch size 8 (Proposed framework) (f) Batch size 32 (Proposed framework)

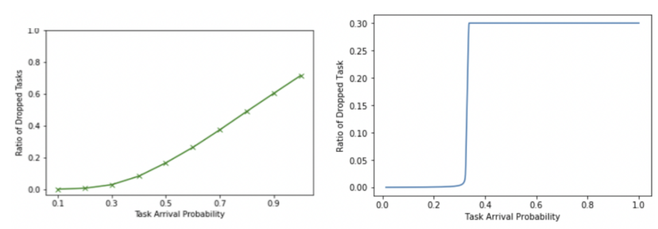
Convergence under different batch sizes.

Results: Delay & Dropped Tasks



(a) Average delay vs task arrival probability (state-of-the-art) (b) Average delay vs no. of devices (state-of-the-art) (c) Average delay vs task arrival probability (proposed framework) (d) Average delay vs no. of devices (proposed framework)

Average delay vs. TAP and number of devices.



(a) State-of-the-art Solution (b) Proposed Solution

Dropped task ratio vs. TAP.